

AIエージェントコーディング時代のブランチマージ

42ブランチのフィーチャー群を、段階的なPR列（PRトレイン）へと計画・分離・リベースした実践のケーススタディ — Guaardvarkの **macOS / Apple Silicon** 拡張を題材に。

- リポジトリ: `guaardvark/guaardvark` (upstream) ↔ `Kwik-Dev/guaardvark` (fork)
- 執筆時点の基準ブランチ: `pr/m2-lora-trainer-mps`
- ツール: GitButler (`but`)、git worktree、GitHub PR、Herdrベイン内で動作する AIコーディングエージェント (`pi`)
- 日付: 2026-09-11

1. なぜこれが新しい問題なのか

AIエージェントは、整った1本のフィーチャーブランチを生み出さない。エージェントは **小さく速いコミットの連続** を生み出し、直前の文脈を頭に保持し続けるため、それらが **しばしば互いに積み重なる**。Guaardvarkでの1回のエージェントセッションは、**42本のバーチャルブランチ**を蓄積した **クリーンなもの**、**累積的なもの**（他のフィーチャーブランチを土台にしたもの）、**保留中のものが混在していた**。

エージェント駆動開発では作業の単位は **コミットの連続** だが、upstreamのレビュアーは依然としてレビューの単位を **単一の貫いたPR** に求める。以下のワークフロー全体は、この2つの形を折り合わせるためのものである。

従来の `git merge` をここで使えない理由は3つある。

- 累積ブランチ**。 `fix/vision-sync` は `feat/llm-providers` を土台にしていた。これをマージすると、provider機能全体を再び引き込んでしまう。
- 動き続けるupstream**。PRトレインを構築している *最中* に `upstream/main` が `23ab76e3` → `4b9763ee` へ進み、すべてのスナップショットのベースを無効化した。
- トラック間でファイルが重複**。 `start.sh`、`real_trainer.py`、`llm_service.py` はそれぞれ *別々の* トラックのブランチから触られており、統合の順序が重要になる。

2. Stage 0 — エージェントにGitButlerで `feat/*` ブランチを作らせる

GitButlerのモデル（1つの作業ディレクトリに適用されるバーチャルブランチ）は、エージェント **作業と相性が良い**。エージェントはブランチごとにworktreeを必要とせず、**論理的な** ブランチへコミットし続けられる。 `but status` はワークスペースをツリーとして表示する。

```
└─ at [feat/lora-training-fixes]
  ● ppkx feat(lora): facial-hair keep in cast desc, job_id passthrough, pod path guard
  └─
└─ ru [feat/runpod-lora-trainer]
  ● qvq feat(lora): pass job_id through remote trainer + progress ETA
  ● spp feat(lora): S3/R2 input staging, output bucket+prefix, ...
  ● tvr fix(lora): pod Dockerfile — modern CUDA 12 base + bundle trainer scripts
```

`but branch list` は **applied**（ワークスペース内）と **unapplied** のブランチを 分離する。これが素材である。42本の適用済みバーチャルブランチに加え、すでに切られた `pr/*` スナップショットがunappliedとして存在する。

導かれたルール: エージェントのブランチは **マージ単位** ではなく **ドラフトコミット** として扱う。upstreamへ直接マージしない。まず、レビュアー向けのストーリーに対応する **名前**を与える。

git worktree と GitButler の比較

どちらも「同時に複数のブランチを扱いたい」を解決するが、解決の方向は逆である。worktreeは各ブランチに **ディスク上の専用チェックアウト** を与える。GitButlerは 多数のブランチを **1つの作業ディレクトリ** にバーチャルブランチとして同居させる。

観点	git worktree	GitButler
モデル	1ブランチ = 1ディレクトリ (<code>git worktree add</code>)	多数のバーチャルブランチを単一の作業ツリーに適用
分離	強 — 別ファイルシステム、別 <code>node_modules/venv</code>	弱 — 同一ファイル。変更はhunk単位でブランチに振り分け
並行性	真の並列チェックアウト。M2のテストを走らせつつM3を編集	直列 — 作業ツリーは1つ、よって同時に1操作のみ
エージェント適性	ブランチごとにworktreeが必要。セットアップが重い	新しいディレクトリなしで 論理的 ブランチへコミットし続けられる
切替コスト	別ディレクトリへ <code>cd</code> （安価、チェックアウト不要）	<code>but branch</code> の <code>apply/unapply</code> （安価、チェックアウト不要）
ブランチの積み重ね	自然 — 1つのworktreeで積み、別で分岐	自然 — <code>but status</code> が積み重なりを視覚表示
ベース陳腐化時のリベース	worktreeごとに <code>git rebase</code>	内部は同じ <code>git rebase</code> 。GitButlerがUIを被せる
ディスク/依存コスト	高 — Nチェックアウト × 依存	低 — チェックアウトは1つ
並列マージ	同一リポジトリなら依然危険 — インデックス破損	作業ツリーが1つなので、構造的に同時1操作のみ
向く用途	長期の並列ビルド、2状態の同時テスト	小さなブランチを量産する高速なエージェント
弱点	worktreeごとに依存/ツールが増え、乖離しやすい	作業ツリーが1つなので真の並列テスト実行は不可

このケースでの使い分け:

- GitButler** は **オーサリングの中心地** (*authoring surface*) — エージェントが生んだ42本の `feat/*` ブランチを、1つのワークスペース内のバーチャルブランチとして扱う場所である。 `but status` のツリーや `but branch list` の `applied/unapplied` を見ながら、ブランチの **作成・整理・再編成** を行う。ブランチごとに別ディレクトリや依存を作る必要はない。これは「大量の素案を並べて、最終形に整えるための作業場」である。

- **git worktree** は 非常口 (escape hatch) — 使い捨ての **worktree** を切って、M3の ComfyUI環境をそのまま適用した状態でPR-182の証拠を生成した。しかも **M2のコードには一切触れずに**、別のGitツリーで検証できる。これは「本流の作業を壊さずに、別の 独立した検証を走らせたい」ときに使う、worktreeの本領である。

両者に共通する問題: 同一リポジトリ内で **git merge** を並列実行するのは 依然として危険である。GitButlerの単一作業ソリーは構造上これ無害化するが、 worktreeではマージを直列に保つ必要がある (§7参照)。

経験則: GitButlerで形を整え (エージェントのコミット連続を論理ブランチへ)、「**真に独立した検証が必要ときだけ worktreeを使う**」 — あるブランチを無傷に保ちつつ 別ブランチをクリーンにテストする、あるいはPRヘッドを乱してはならない証拠を生成する、 といった場面である。

3. Stage 1 — 段階的PR - 計画・整理

42本のブランチは **2つのトラック** に分類し、各PRが1つのテーマを担い、レビュアーが 順序を追えるようにした。

macOSトラック — Apple Siliconの基盤 (先に着地)

PR	ブランチ	根拠
M1	feat/audio-mps-whisper-filmcrew, fix/mps-video-unload	MPSオーディオ + whisper.cpp STT + ビデオunload修正
M2	feat/lora-trainer-mps	MPSでのLoRA学習
M3	feat/zimage-comfyui, feat/zimage-comfyui-mps, feat/imagemodel-comfyui	Apple SiliconでのComfyUI経由Z-Image
M4	feat/llm-providers, voice-openai-routing	OpenAI互換ルーティング (CUDA非搭載 Mac向けクラウドfallback)

一般トラック — プラットフォーム非依存

PR	テーマ
G1	Film Crew + オーディオ/ビデオの磨き込み、i2v、キャプション
G2	任意のMCPサーバ起動／クリーンアップ
G3	RunPodリモートLoRAトレーナー + タイムアウト修正
G4	キャストのショット数 + チャットルーティング
G5	Vision/LLM/OpenAIルーティング (M4に積層)
G6	折りたたみアラートUI + MUI ref対応
G7	エージェント設定 + ドキュメント + gitignore

依存マップ(一つの成果物)

何かを切る前に、両トラックから触られるファイルを記録する。

ファイル	macOSブランチ	一般ブランチ
start.sh / stop.sh	M1	G2
backend/tasks/production_swarm_tasks.py	M1	G5
backend/tasks/lora_trainer_tasks.py	M2, M3	G3
backend/tools/image_tools.py	M3	G4
backend/utlis/llm_service.py	M4	G5
plugins/lora_trainer/real_trainer.py	M2	G3

この表が そのまま マージ順である。macOSが基盤として先に着地し、一般トラックが その上に着地する。トラック間の衝突はその時点で解決される。

順序の原則: PRは ブランチではなく ストーリー である。ユーザーに見える テーマでグループ化し、エージェントが たまたまコミットした順ではなく、依存関係で 順序づける。

4. Stage 2 — スナップショット: PRごとに1本のクリーンなブランチ

決定的な判断 (2026-09-08に再設計) は、各PRグループを **upstream/main** から切った 独立したスナップショットブランチ にすることだった — dev への累積マージでは ない。

各スナップショットは **自グループの内容のみ** を含むため、upstream/main に 対するPR差分はまさにそのフィーチャーだけになり、以前のPRからの持ち越しがない。

クリーンなスナップショットを切る

```
git checkout -b pr/<name> upstream/main
git merge --no-ff <feat-branch> -m "<group>: <branch>"
# 競合を手作業で解決した後:
git add <resolved-files> && git commit --no-edit
git push -u origin pr/<name>
```

累積スナップショットを切る (固有コミットをcherry-pick)

feat/* ブランチが別のフィーチャーの上に載っている場合、マージするとベースを 再び引き込んでしまう。固有のコミットを見つけ、それだけをcherry-pickする。

```
git log upstream/main..<branch> --oneline # 固有コミットを特定
git checkout -b pr/<name> upstream/main
git cherry-pick <unique-commit-1> <unique-commit-2> ...
git cherry-pick --continue # 解決後
git push -u origin pr/<name>
```

例 **G5** では、fix/vision-sync、chore/llm-providers-followup、feat/filmcrew-openai-compatible はすべてM4の openai_provider.py / ModelManagementSection.jsx に依存するため、

G5の差分は `upstream/main` ではなく **M4**に対して クリーンになる。

スナップショットがマージ可能か検証する

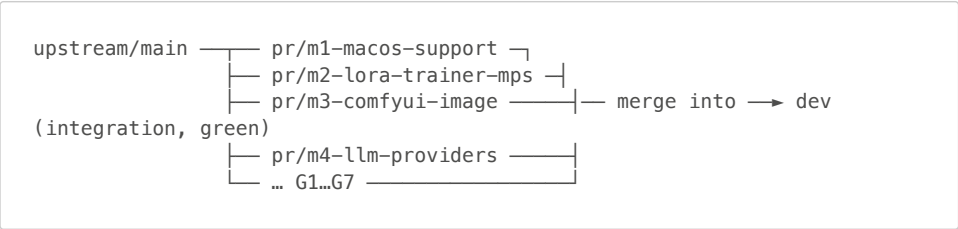
```
git merge-base --is-ancestor upstream/main pr/<name> && echo MERGEABLE
git diff --stat upstream/main..pr/<name>
```

5. Stage 3 — 統合ブランチとしての dev

`dev` はfork内部の 統合 ブランチである。upstream向けブランチとしては使用しない。PR前に、グリーンであることを証明する場所である。

- 11本のトラックのスナップショット（M1 → M2 → M3 → M4 → G1 → ... → G7）が 順番に `dev` へマージされる。
- `dev` は グリーン を検証済み: フロントエンド `eslint` クリーン、`vitest` 230/230通過、Python構文クリーン、競合マーカーなし。
- forkへforce-pushされている（ベース比 `↑142`）。

`dev` はスナップショットの下流であり、上流ではない。



インテグレーションの段階で トラック間の競合が解決される。G1..G7のトラックが m1..m4 トラックの上にスタックされ、依存マップ上で重複するファイルのコンフリクトが解決されれば、その修正 `dev` で一度にまとめれる。

統合時(インテグレーション)で表面化した修正

- `run_acestep.py` に コミット済みの競合マーカー があった — `dev` と `pr/m1-macos-support` の両方で修正 (`_pick_device()` を使用)。
- `buildPlanRequest.test.js` はG1の新フィールド (`respect_bin_order / customer_context / caption_defaults`) 向けに更新が必要だった。
- `remark-gfm` は `package.json` にあったが `node_modules` から欠けていた。
- `plugins/training/scripts/finetune_model.py` は `upstream/main` に 既存の 構文エラー（未終端の三重クォートf-string）がある — 我々のものではないため放置。

教訓: 統合ブランチは、「クリーン」なはずのスナップショットがそうでないことを 発見する場所である。それを見込んでおくこと。

6. Stage 4 — PRとupstreamが共に動く 中で主流をリベースする

これがエージェント時代のマージを本当に難しくする部分である: このマージは**Upstram**から、と**PR**からの両方が発生する

Upstram/main

スナップショットを切った 後に `upstream/main` が `23ab76e3` → `4b9763ee` と更新 そのため、すべての `pr/*` ブランチは古い `main` をベースにしている。スナップショット 戦略のおかげでこれは 回復可能 である。各スナップショットは小さく自己完結した差分 なので、新しい `main` へのリベースは限定的な競合であり 全体を蒸し返すこと にはならない。

```
# forkのmainを同期した上で、devと各スナップショットを再設定
git fetch upstream
git checkout dev && git rebase upstream/main
# 差分がなお適用できる各スナップショットを再カットまたはリベース
git checkout pr/<name> && git rebase upstream/main
git push --force-with-lease origin pr/<name>
```

動くPR: 単一コミットの所有権が2度変わった

最も明快な例は、M2 / G3作業中の タイムアウト修正 である。同じ論理変更が、1セッションの間に異なるPRを渡り歩いた。

- 当初は **G3** スナップショットの一部だった (`pr/g3-runpod-lora-trainer`、squash済み `80da48a4`)。
- M2**から欠けている ことが判明 — つまり本当に必要とするPRに入っていなかった。
- オーナーの決定により **M2**専属に再割り当て:
 - `pr/m2-lora-trainer-mps` は `e9593922` (daemon 3時間 / load 1時間)、`2e2ffa83` (タスク上限255分)、`8f2dd1d0` (fail loudly/reconcile)、`4f902ba2` (テスト) を保持。
 - `pr/g3-runpod-lora-trainer` はそれら なしで再構築 され、force-pushで `80da48a4` → `68fe8451` に (RunPod作業は無傷。`real_trainer.py` は差分から除外。reaperは45分に戻した)。

```
Before:  G3 = { RunPod + timeouts }      M2 = { MPS LoRA }
After:    G3 = { RunPod }                  M2 = { MPS LoRA + timeouts }
```

両方のPRが変更され、両方のベースが動き、`upstream/main` が両者の下で動いた。 エージェントが高速に反復するとき、これは例外ではなく通常のケースである。

PR ブランチ (スナップショット)

レビュー用の証拠は、陳腐化した成果物ではなく 現在の PRコードに追隨する必要が あった。

- 初期の **8月27日のrun B** 成果物は 陳腐化として却下 された — オーナーは 現在のPRコードからの証拠を求めた。
- 現在のM2ヘッド (`4f902ba2`、Elara subject 1、400ステップ、768解像度、11画像) での 新規run が代替とされたが、その後オーナーの指示により **step 51/400で停止** され、承認なしに再開してはならない。

- MPSで学習済みLoRAを証明する画像は、GitHubが描画できるよう **別ブランチ evidence/pr-182** へ pushし、PRブランチの差分をクリーンに保った。

教訓: 証拠は ブランチ であり、それ自体が動いているPRヘッドに固定されなければ ならない。PRブランチからは切り離しておくこと。

7. 要点

1. エージェントに自由にコミットさせる — GitButlerのバーチャルブランチへ。コミットの連続と戦わない。
2. まず**PRマップ**を計画する — テーマ + 依存表 — ブランチを切る *前に*。
3. **PRごとに1本のクリーンなスナップショットを `upstream/main` から切る**。累積ブランチの固有コミットは cherry-pickする。
4. **すべてを `dev` で統合する** — 依存順に。グリーンを証明し、トラック間の競合を *そこで* 解決する。
5. ベースが動くことを前提とする。 `dev` と各スナップショットを新しい `upstream/main` へ `--force-with-lease` でリベースする。
6. **PRの所有権が動くことを前提とする**。変更はエージェントが最初に置いたPRとは 別のPRに属することがある。影響を受けるスナップショットを再構築しforce-pushする。
7. 証拠は**専用ブランチ**に置き、現在のPRヘッドに固定する。
8. **PRは一度に1つずつ**、順番に提出する (M2 → M3 → M4 → G1 → ... → G7)。

学んだ落とし穴

- **git merge** を並列実行しない。1つのworktreeで2つのマージを同時に行うと インデックスが破損する。1つずつ。
- マージ/**cherry-pick**での **git checkout --theirs** は */!dev* の版を取る — これはupstreamの機能を欠いている。手作業で解決する。upstreamの構造を保ち、 そのブランチの実際の変更のみを加える。
- 一部の **feat/*** は累積的。まず **git log upstream/main..<branch>** を実行し、別のフィーチャーを含むならcherry-pickする。
- **pr/*** は **dev** のスナップショットではない。 `dev` はそれらをマージする統合 ブランチであり、各 **pr/*** は `upstream/main` から切られる。

8. 提出

各PRを一度に1つずつ:

1. `https://github.com/guaardvark/guaardvark/compare/main...Kwik-Dev:pr/<name>` を開く。
2. `base = guaardvark:main、head = Kwik-Dev:pr/<name>` を確認する。
3. **"Able to merge"** と表示されることを確認する。
4. タイトル + 説明を書く (追跡issueをリンクする)。
5. 提出する。

CLA: 最初のPRでCLA Assistant Liteが起動する。要求された1行コメントで署名する。

認証: PRを開くには、有効な `GITHUB_TOKEN` ではなく、keyringトークン (フル `repo` スコープ) が必要である。

9. まとめ

- エージェント駆動開発では、**コミットの連続とレビュー単位が乖離する**。スナップショットブランチのパターンがその橋渡しである。エージェントのコミットの 連続を、クリーンで独立にレビュー可能なPRの列へと作り替える。
- 統合は後付けではなく **想定されたステージである**。 `dev` は、レビューアが目にする 前にトラック間の衝突を吸収するために存在する。
- リベースは一回限りの出来事ではなく**継続的である**。 エージェントが高速に コミットしupstreamが動いたため、ベースとPR内容の双方が変わる。ワークフローは `--force-with-lease` が日常であることを前提とすべきである。
- 計画 (依存表) は**ブランチより長生きする**。 ブランチは再構築・squash・force-pushされるが、*順序* こそが安定して残る。

付録 — ブランチマップ (執筆時点)

スナップショット	ベース	内容	PR
<code>pr/m1-macos-support</code>	<code>upstream/main</code>	MPSオーディオ/whisper/ビデオ	#154 (マージ済み)
<code>pr/m2-lora-trainer-mps</code>	<code>upstream/main</code>	MPSでのLoRA学習 + タイムアウト	#182
<code>pr/m3-comfyui-image</code>	<code>upstream/main</code>	ComfyUI Z-Image / imagemodel	□
<code>pr/m4-llm-providers</code>	<code>upstream/main</code>	LLMプロバイダ + voiceルーティング	□
<code>pr/g1-audio-video-polish</code>	<code>upstream/main</code>	オーディオ/音楽の磨き込み、i2v、filmcrew	□
<code>pr/g2-mcp-start</code>	<code>upstream/main</code>	MCPサーバ起動/クリーンアップ	□
<code>pr/g3-runpod-lora-trainer</code>	<code>upstream/main</code>	RunPod LoRAトレーナー	□
<code>pr/g4-cast-shot-count</code>	<code>upstream/main</code>	キャストのショット数 + チャットルーティング	□
<code>pr/g5-vision-llm-routing</code>	<code>pr/m4-llm-providers</code>	vision同期 + llmフォローアップ	□
<code>pr/g6-collapsible-alerts</code>	<code>upstream/main</code>	折りたたみアラートUI	□
<code>pr/g7-agent-config-docs</code>	<code>upstream/main</code>	エージェント設定 + ドキュメント	□
<code>evidence/pr-182</code>	—	描画用の証拠画像	—